Leer un EXPLAIN ANALYZE en PostgreSQL

Qué es lo mejor y qué es lo peor que te puede aparecer — y qué hace cada cosa

Material de cátedra · Bases de Datos · Vale para PostgreSQL 12 en adelante (los detalles menores del formato cambian entre versiones)

Cuando PostgreSQL recibe una consulta no la ejecuta "tal cual está escrita". El **planificador** (*planner*) evalúa varias formas posibles de resolverla —qué tabla leer primero, si conviene usar un índice, con qué algoritmo hacer cada join— y elige la que **estima** más barata. **EXPLAIN** muestra ese plan. Este documento explica cómo leerlo, qué hace cada operación que puede aparecer, y cuáles son las señales de que un plan es bueno o malo.

1. EXPLAIN vs. EXPLAIN ANALYZE

- **EXPLAIN** — muestra el plan elegido, con **estimaciones** (costo y cantidad de filas). **No ejecuta** la consulta.
- **EXPLAIN ANALYZE** — **ejecuta realmente** la consulta y muestra, al lado de cada estimación, lo que pasó de verdad: tiempo en milisegundos, filas reales y cantidad de iteraciones. Es el que sirve para diagnosticar.

> **Cuidado**
>
> **EXPLAIN ANALYZE ejecuta la consulta.** Si es un INSERT, UPDATE o DELETE, los cambios se aplican de verdad. Para medirlos sin riesgo, envolverlos en una transacción y deshacerla:
>
> ```
> BEGIN;
> EXPLAIN ANALYZE DELETE FROM linea_factura WHERE nro_factura = 10;
> ROLLBACK;
> ```

Opciones útiles

```
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;
```

Opción	Qué agrega
`ANALYZE`	Ejecuta y muestra tiempos, filas e iteraciones reales.
`BUFFERS`	Cuántas páginas de 8 KB leyó de la caché (*hit*), del disco (*read*) y de archivos temporales (*temp*). Conviene incluirla siempre; desde PostgreSQL 18 se activa sola junto con ANALYZE.
`VERBOSE`	Muestra qué columnas devuelve cada nodo (`Output:`) y nombres calificados.
`FORMAT JSON`	Salida en JSON, para pegar en herramientas visuales.
`COSTS OFF`, `TIMING OFF`	Ocultan los costos estimados / los tiempos por nodo. Útiles para simplificar la lectura o para que la medición de tiempo no distorsione consultas muy rápidas.

Herramientas: **explain.dalibo.com** y **explain.depesz.com**. Se pega el texto del plan y lo muestran como árbol, con colores según dónde se va el tiempo. Muy recomendables para planes largos.

2. Anatomía de una línea del plan

Cada nodo del plan ocupa una línea principal (más algunas líneas de detalle debajo). Esta es una típica, con EXPLAIN (ANALYZE, BUFFERS):

```
Index Scan using idx_lf_producto on linea_factura l  (cost=0.43..45.61 rows=12 width=24)
                                                     (actual time=0.031..0.084 rows=12 loops=1)
  Index Cond: (id_producto = 15)
  Buffers: shared hit=15
```

(En la salida real el paréntesis de cost y el de actual van en la misma línea; acá se cortó por espacio.)

Fragmento	Qué significa
`Index Scan using idx_lf_producto on linea_factura l`	El **tipo de nodo** (la operación), el índice que usa y la tabla que lee, con su alias.
`cost=0.43..45.61`	Costo **estimado**: **de arranque** (lo que cuesta antes de devolver la primera fila) **.. total**. Son unidades arbitrarias —leer una página secuencial vale 1—, **no son milisegundos**. Sirven para que el planificador compare planes entre sí.
`rows=12`	Filas que el planificador **estima** que va a devolver este nodo.
`width=24`	Ancho promedio estimado de cada fila, en bytes.
`actual time=0.031..0.084`	Tiempo **real** en milisegundos: hasta la primera fila **..** hasta la última. Es **por iteración** (ver loops). Incluye el tiempo de todos los nodos hijos.
`rows=12` (en actual)	Filas **reales** devueltas, promedio por iteración.
`loops=1`	Cuántas veces se ejecutó el nodo. El tiempo y las filas están **divididos por loops**: para el total hay que multiplicar. `rows=1 loops=5000` son 5.000 filas.
`Index Cond` / `Filter`	**Index Cond**: la condición se resolvió con el índice (sólo se visitaron las entradas que cumplen). **Filter**: se leyeron las filas y *después* se descartaron las que no cumplen.
`Rows Removed by Filter`	Cuántas filas se leyeron en vano (se leyeron, se evaluó la condición y se descartaron). Un número grande comparado con las filas devueltas es la señal de alarma más común.
`Buffers: shared hit=15`	**hit**: páginas encontradas en la caché de PostgreSQL. **read**: leídas desde afuera (disco o caché del sistema operativo). **temp read/written**: archivos temporales — algo no entró en memoria.
`Planning Time` / `Execution Time`	Al final del plan. Cuánto tardó en armar el plan y cuánto en ejecutarlo. Execution Time es el número que se compara antes y después de una mejora.

Dos ideas para no olvidar

- **Izquierda vs. derecha.** El paréntesis de `cost` es lo que el planificador *creía* que iba a pasar; el de `actual` es lo que *pasó*. Cuando las filas estimadas y las reales difieren mucho, el planificador eligió el plan con información incorrecta.
- **Los tiempos se acumulan hacia arriba.** El `actual time` de un nodo incluye el de sus hijos. El nodo raíz (la primera línea) muestra el tiempo de toda la consulta.

3. Cómo se lee el árbol

El plan es un **árbol**. Cada línea que empieza con `->` es un hijo del nodo que está más arriba con menos indentación. Los datos fluyen **de los hijos al padre**: los nodos más indentados (las hojas) leen tablas; los de arriba combinan, filtran, ordenan y agregan. Se lee **de adentro hacia afuera**: primero lo más indentado, último la primera línea.

```
Sort  (cost=...) (actual time=8.90..8.91 rows=12 loops=1)            <- 3.º ordena el resultado
  Sort Key: f.fecha
  Sort Method: quicksort  Memory: 26kB
  ->  Hash Join  (cost=...) (actual time=1.20..8.80 rows=12 loops=1)  <- 2.º combina las dos tablas
        Hash Cond: (l.nro_factura = f.nro_factura)
        ->  Seq Scan on linea_factura l  (...)                      <- 1.º lee las líneas
              Filter: (id_producto = 15)
        ->  Hash  (...)                                             <- 1.º arma la tabla hash
              ->  Seq Scan on factura f  (...)                      <-     ... con las facturas
```

En un join, el **primer hijo** listado es el lado **externo** (*outer*) y el **segundo** es el **interno** (*inner*). Para encontrar el problema de un plan, se busca el nodo cuyo tiempo propio (su `actual time` × `loops`, descontando el de sus hijos) es el más grande: ahí es donde se va el tiempo.

4. Nodos de acceso a datos: ¿cómo llega a las filas?

Son las hojas del árbol. Hay cuatro formas principales de leer una tabla y la elección entre ellas es, casi siempre, lo que define si un plan es bueno o malo.

Seq Scan
Lectura secuencial de la tabla completa

Qué hace	Lee **toda** la tabla, página por página, de principio a fin, y evalúa el `Filter` sobre cada fila. No usa ningún índice.
Está bien cuando	• La tabla es chica (unas pocas páginas): un índice no ahorraría nada. • La consulta necesita gran parte de la tabla (más de ~5–10 % de las filas, según el caso) o no tiene WHERE. • Es el hijo de un Hash Join o de un Sort que de todos modos procesa todo.
Señal de alarma cuando	• La tabla es **grande**, hay un `Filter` selectivo y `Rows Removed by Filter` es **enorme** comparado con las filas devueltas. Ejemplo: devuelve 214 filas y descartó 1.999.786. Es un índice que falta, o que existe pero no se puede usar. • Aparece con `loops` alto (como hijo interno de un Nested Loop): se relee la tabla entera en cada vuelta.
Por qué a veces no usa un índice que existe	• Hay una **función o expresión sobre la columna**: `WHERE upper(nombre) = 'X'`, `WHERE date(fecha) = ...`, `WHERE precio * 1.21 > 100`. El índice está sobre la columna, no sobre la expresión. • **Tipos distintos**: columna `integer` comparada con un texto, o `varchar` con `integer`. • `LIKE '%algo'` (comodín al principio): un índice B-tree no sirve. • La condición **no es selectiva**: trae casi toda la tabla y el Seq Scan es más barato. • **Estadísticas viejas**: falta `ANALYZE tabla` después de una carga masiva. • La tabla es demasiado chica para que valga la pena.

Index Scan

Búsqueda por índice, con acceso a la tabla por cada fila

Qué hace	Recorre el índice buscando las entradas que cumplen la `Index Cond` y, por cada una, salta a la tabla (el *heap*) a buscar la fila completa. Devuelve las filas **en el orden del índice**.
Está bien cuando	• La condición es selectiva: devuelve pocas filas respecto del total. • Hay `ORDER BY` sobre la columna del índice: ahorra un Sort completo. • Es el lado interno de un Nested Loop (búsquedas puntuales repetidas).
Señal de alarma cuando	• `rows` es muy grande: miles o millones de saltos aleatorios a la tabla pueden ser **más lentos** que un Seq Scan. Si el planificador lo eligió igual, suele ser porque estimó mal (comparar rows estimadas vs. reales). • Además de la `Index Cond` aparece un `Filter` con `Rows Removed by Filter` grande: el índice sólo resolvió *parte* de la condición. Quizás falta una columna en el índice (índice compuesto).
Qué mirar	`Index Cond` (¿qué parte del WHERE resolvió el índice?), `rows` real vs. estimada, `loops` si está bajo un Nested Loop.

Index Only Scan

El índice alcanza: no hace falta ir a la tabla

Qué hace	Igual que Index Scan, pero **todas las columnas** que necesita la consulta están en el índice, así que no salta a la tabla… salvo cuando necesita verificar si la fila es visible (por MVCC). Cada una de esas verificaciones se cuenta en `Heap Fetches`.
Lo mejor que puede aparecer	`Index Only Scan` **con** `Heap Fetches: 0`. Es la forma más barata que existe de responder una consulta con muchas filas: se lee sólo el índice, que es más compacto que la tabla.
Cuándo aparece	• El SELECT pide sólo columnas que están en el índice (o en su cláusula `INCLUDE`). • `SELECT count(*) FROM tabla WHERE columna_indexada = ...`.
Señal de alarma cuando	`Heap Fetches` es alto (cercano a `rows`): el *mapa de visibilidad* está desactualizado y termina yendo a la tabla igual. Se resuelve con `VACUUM tabla` (el autovacuum lo hace solo con el tiempo).

Bitmap Index Scan + Bitmap Heap Scan

El índice en dos pasos: primero marcar, después leer

Qué hace	**Bitmap Index Scan** recorre el índice y marca en un mapa de bits en memoria *qué páginas* de la tabla tienen filas candidatas. **Bitmap Heap Scan** lee esas páginas **en orden físico** (sin saltos aleatorios) y vuelve a evaluar la condición (`Recheck Cond`). Siempre aparecen juntos, el Index Scan como hijo del Heap Scan.
Está bien cuando	• La condición devuelve una cantidad **intermedia** de filas: demasiadas para un Index Scan (muchos saltos), pocas para un Seq Scan. • Hay condiciones con `OR` / `AND` sobre columnas con índices distintos: aparecen `BitmapOr` / `BitmapAnd` combinando varios índices.
Señal de alarma cuando	`Heap Blocks: exact=... lossy=...` con `lossy` grande y `Rows Removed by Index Recheck` alto: el mapa de bits no entró en `work_mem` y degradó a marcar *páginas* en vez de *filas*, así que relee y refiltra mucho de más.

Otros scans que pueden aparecer

Nodo	Qué hace
`CTE Scan`	Lee el resultado de un `WITH` que se materializó (se calculó una vez y se guardó).
`Subquery Scan`	Lee el resultado de una subconsulta en el FROM que no se pudo “aplanar” dentro de la consulta principal.
`Function Scan`	Lee lo que devuelve una función que produce filas (`generate_series`, `unnest`, funciones propias). El planificador no sabe cuántas filas devuelve: las estimaciones suelen ser malas.
`Values Scan`	Lee una lista `VALUES (...), (...)` escrita en la consulta.
`TID Scan`	Acceso directo por `ctid` (posición física). Raro fuera de mantenimiento.

5. Nodos de join: ¿cómo combina dos tablas?

PostgreSQL tiene tres algoritmos de join. Ninguno es “el bueno”: cada uno gana en un escenario distinto, y el planificador elige según cuántas filas espera de cada lado y qué índices hay disponibles.

Nested Loop
Por cada fila de un lado, buscar en el otro

Qué hace	Por **cada fila** del hijo externo (el primero listado) ejecuta **completo** el hijo interno. Por eso el interno aparece con `loops` igual a la cantidad de filas del externo.
Está bien cuando	El externo es **chico** (pocas filas) y el interno es un **Index Scan** rápido: pocas vueltas × acceso barato. Es el join ideal para “traer las líneas de esta factura” o “la factura de cada una de estas 200 líneas”.
Lo peor	• Externo con **muchas** filas e interno con **Seq Scan**: N × M. Si aparece un `Seq Scan` con `loops=50000`, ahí está el problema: la tabla se leyó entera 50.000 veces. • Nested Loop sin condición de join (sin `Join Filter` ni Index Cond que lo vincule): producto cartesiano.
Qué mirar	Un nodo `Materialize` debajo del interno significa que PostgreSQL guardó su resultado en memoria para no recalcularlo en cada vuelta: es una optimización, no un problema.

Hash Join
Armar una tabla hash con un lado y buscar en ella con el otro

Qué hace	Toma el hijo que está debajo del nodo `Hash` (el planificador intenta que sea el más chico), lo lee **entero** y arma una tabla hash en memoria con la clave del join. Después recorre el otro hijo **una sola vez** y busca cada fila en la tabla hash.
Está bien cuando	Hay que juntar dos conjuntos grandes por **igualdad** (`=`) y no hay un índice útil, o cuando los Seq Scan son inevitables. `Batches: 1` significa que la tabla hash entró completa en memoria (`work_mem`): es el caso feliz.
Señal de alarma cuando	`Batches: 8` (cualquier valor mayor que 1): la tabla hash **no entró** en `work_mem` y se partió en archivos temporales en disco; aparecen `Buffers: temp read/written`. Soluciones: subir `work_mem` para esa sesión, o reducir las filas antes del join (filtrar primero, índice).
Limitación	Sólo sirve para joins de igualdad. Con `<`, `>`, `BETWEEN` el planificador tiene que usar Nested Loop o Merge Join.

Merge Join Dos listas ordenadas recorridas en paralelo	
Qué hace	Necesita que **ambos lados vengan ordenados** por la clave del join (los ordena con un `Sort` si hace falta, o usa un Index Scan que ya los entrega ordenados) y los recorre en paralelo, como un cierre relámpago.
Está bien cuando	Los dos lados ya vienen ordenados por índice, los conjuntos son grandes, o el resultado tiene que salir ordenado por esa misma clave (ahorra el Sort final).
Señal de alarma cuando	Necesita un `Sort` grande de cada lado, y ese Sort cae a disco. El costo real está en los Sort, no en el join.

Join	Cuándo lo elige el planificador	Costo aproximado
`Nested Loop`	Externo chico + índice en el interno. También para tablas mínimas y para joins sin igualdad.	N × (costo de una búsqueda interna)
`Hash Join`	Igualdad, sin índice útil, tablas medianas o grandes.	N + M, si la hash entra en memoria
`Merge Join`	Ambos lados ordenados (por índice o Sort), conjuntos grandes.	N + M, más el costo de ordenar

6. Otros nodos frecuentes

Aparecen por encima de los scans y de los joins: ordenan, agrupan, limitan, eliminan duplicados o reparten el trabajo entre procesos. En general no son el origen de un problema, pero muestran sus síntomas: un Sort que cae a disco está diciendo que le llegaron demasiadas filas.

Nodo	Qué hace	Bien / Mal
`Sort`	Ordena su entrada (ORDER BY, o para preparar un Merge Join / GroupAggregate).	**Bien:** `Sort Method: quicksort Memory: 25kB` o `top-N heapsort` (ORDER BY + LIMIT: sólo guarda las N mejores). **Mal:** `external merge Disk: 45000kB` — no entró en `work_mem` y escribió a disco. Un índice sobre la clave del ORDER BY puede eliminar el Sort por completo.
`Incremental Sort`	Aprovecha que la entrada ya viene ordenada por las primeras columnas de la clave y sólo ordena el resto.	Generalmente bien: es más barato que un Sort completo.
`Aggregate` `HashAggregate` `GroupAggregate`	Calculan SUM, COUNT, AVG, etc. **Aggregate**: sin GROUP BY. **HashAggregate**: agrupa con una tabla hash en memoria. **GroupAggregate**: necesita la entrada ordenada por las columnas del GROUP BY (mirar si hay un Sort debajo). `Partial` / `Finalize` son las versiones para consultas paralelas.	**Mal:** HashAggregate con `Batches` > 1 o `Disk Usage`: demasiados grupos para la memoria.
`Hash`	Hijo de un Hash Join: construye la tabla hash.	Mirar `Batches` (debe ser 1) y `Memory Usage`.
`Limit`	Devuelve las primeras N filas y **detiene** a sus hijos.	**Bien:** directamente sobre un Index Scan ordenado: lee sólo N entradas. **Ojo:** si debajo hay un Sort o un HashAggregate, igual hay que procesar *todo* antes de cortar.
`Unique`	Elimina duplicados de una entrada ordenada (DISTINCT, UNION).	Mirar el Sort que suele tener debajo.

Nodo	Qué hace	Bien / Mal
`Materialize`	Guarda el resultado del hijo en memoria para releerlo varias veces (típico bajo un Nested Loop).	Optimización; sólo preocupa si el resultado guardado es enorme.
`Append` `Merge Append`	Concatena resultados de varios hijos: UNION ALL, tablas particionadas.	Con particiones, verificar que *no* aparezcan todas: el WHERE debería descartar particiones (*pruning*).
`Gather` `Gather Merge`	Consulta paralela: reúne lo que producen varios procesos *worker*. Los tiempos de los hijos son por worker.	**Bien:** `Workers Planned: 2, Workers Launched: 2`. **Ojo:** Launched menor que Planned: no había procesos disponibles.
`SubPlan` `InitPlan`	Subconsultas. **InitPlan** se ejecuta **una vez** (no depende de la fila externa). **SubPlan** se ejecuta **por cada fila** (subconsulta correlacionada).	**Mal:** SubPlan con `loops` enorme. Suele arreglarse reescribiendo la subconsulta como JOIN o con `EXISTS`.
`WindowAgg`	Funciones de ventana (`OVER (...)`).	Mirar el Sort que necesita para PARTITION BY / ORDER BY.
`Result`	Nodo trivial: constantes o expresiones sin tabla.	Nunca es un problema.

7. El semáforo: lo mejor, lo aceptable y lo peor

Resumen de qué buscar en un plan. Ninguna señal aislada condena o salva una consulta: el contexto (tamaño de la tabla, cuántas filas se piden) siempre importa.

VERDE — Lo mejor que te puede aparecer

- `Index Only Scan` con `Heap Fetches: 0`.
- `Index Scan` con `Index Cond`, `rows` chico y sin un `Filter` que descarte muchas filas.
- **Estimado ≈ real**: `rows=100` estimadas y `rows=104` reales. El planificador sabía lo que hacía.
- `Hash Join` con `Batches: 1`.
- `Sort Method: quicksort Memory: ...` o `top-N heapsort`: ordenó en memoria.
- `Nested Loop` con externo chico, interno `Index Scan` y `loops` bajo.
- `Limit` directamente sobre un `Index Scan` ordenado, sin Sort en el medio.
- `Buffers`: casi todo `shared hit`, poco `read`, **nada** de `temp`.
- `Execution Time` proporcional a las filas que devuelve: milisegundos para decenas de filas.

AMARILLO — Depende del contexto

- `Seq Scan` sobre una tabla **chica**: es lo correcto. No se “arregla” con índices.
- `Seq Scan` cuando el WHERE trae gran parte de la tabla, o no hay WHERE.
- `Bitmap Heap Scan` sin `lossy`: buen compromiso para una cantidad intermedia de filas.
- Estimación desviada entre 2 y 10 veces: mirar, pero puede no cambiar el plan.
- `shared read` alto en la primera ejecución y `hit` en la segunda: es la caché calentándose. **Ejecutar dos veces** antes de comparar tiempos.
- `Gather` con `Workers Launched` menor que `Planned`.
- `Planning Time` mayor que `Execution Time`: normal en consultas muy rápidas o con muchos joins.

ROJO — Lo peor que te puede aparecer

- `Seq Scan` sobre una tabla **grande** con `Filter` y `Rows Removed by Filter` enorme → falta un índice, o el índice no se puede usar por la forma de la condición.
- `Nested Loop` con `loops` en miles o millones y un `Seq Scan` (o Bitmap Scan) adentro.
- `Sort Method: external merge Disk: ...` → el ordenamiento cayó a disco.
- `Hash Join` / `HashAggregate` con `Batches` > 1 o `Disk Usage`.
- `Buffers: temp read=... written=...` grandes → archivos temporales en disco.
- **Estimado vs. real desviado 100× o más** (`rows=1` estimada, `rows=250000` reales) → estadísticas viejas: `ANALYZE tabla`. O una expresión que PostgreSQL no puede estimar.
- `Index Only Scan` con `Heap Fetches` ≈ `rows` → `VACUUM tabla`.
- `Bitmap Heap Scan` con `lossy` y `Rows Removed by Index Recheck` grande.
- `Filter` (y no `Index Cond`) sobre una columna que **tiene** índice → la expresión impide usarlo.
- `SubPlan` con `loops` enorme → subconsulta correlacionada ejecutada por cada fila.
- Join sin condición entre tablas grandes (producto cartesiano): `rows` = N × M.

8. Por qué “Seq Scan = malo, índice = bueno” es falso

El planificador no elige un índice “porque es lo correcto”: elige el plan de **menor costo estimado**. Leer una tabla secuencialmente es barato por página (lectura contigua, en orden). Un índice implica **saltos aleatorios**: por cada entrada del índice hay que ir a buscar la página de la tabla donde está la fila. Si la consulta necesita el 40 % de la tabla, el índice obligaría a visitar casi todas las páginas en desorden, y probablemente más de una vez: peor que leerlas todas en orden una sola vez.

- Regla aproximada: índice cuando se necesita menos de un ~5–10 % de la tabla; Seq Scan cuando se necesita más. El umbral exacto depende del tamaño de las filas, de la caché y de la configuración (`random_page_cost`, `seq_page_cost`).
- Por eso en tablas de prueba de 20 filas **nunca** se ve un Index Scan: toda la tabla cabe en una página y leerla completa es lo más barato. Para ver la diferencia hace falta volumen: decenas de miles de filas como mínimo.
- El costo depende de las **estadísticas** (cuántas filas tiene la tabla, cuán repetidos son los valores de cada columna). `ANALYZE tabla` las actualiza. Después de cargar muchos datos de golpe, correrlo *antes* de mirar planes; si no, el planificador razona sobre una tabla que ya no existe.

Para probar en clase

Para experimentar en clase se puede forzar al planificador a evitar los Seq Scan (los vuelve carísimos, no los prohíbe):

```
SET enable_seqscan = off;   -- sólo en la sesión actual
EXPLAIN ANALYZE SELECT ...;
RESET enable_seqscan;
```

Sirve para *ver* cuánto peor sería el plan con índice cuando la consulta trae muchas filas. Nunca en producción.

9. Ejemplo completo: de rojo a verde

Esquema de facturación: `factura(nro_factura PK, fecha, id_cliente, total)` y `linea_factura(nro_factura FK, nro_linea, id_producto, cantidad, precio, subtotal)`. Hay 100.000 facturas y 2.000.000 de líneas. La consulta busca en qué facturas y en qué cantidad se vendió un producto:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.nro_factura, f.fecha, l.cantidad
FROM factura f
JOIN linea_factura l ON l.nro_factura = f.nro_factura
WHERE l.id_producto = 15;
```

Plan 1 — sin índice sobre linea_factura.id_producto

```
Hash Join  (cost=3567.00..42841.20 rows=200 width=16) (actual time=31.94..412.71 rows=214 loops=1)
  Hash Cond: (l.nro_factura = f.nro_factura)
  Buffers: shared hit=1250 read=12403
  ->  Seq Scan on linea_factura l  (cost=0.00..38726.00 rows=200 width=12)
                                   (actual time=0.05..380.12 rows=214 loops=1)
        Filter: (id_producto = 15)
        Rows Removed by Filter: 1999786
        Buffers: shared hit=750 read=12000
  ->  Hash  (cost=2317.00..2317.00 rows=100000 width=8) (actual time=31.62..31.62 rows=100000 loops=1)
        Buckets: 131072  Batches: 1  Memory Usage: 4931kB
        ->  Seq Scan on factura f  (cost=0.00..2317.00 rows=100000 width=8)
                                   (actual time=0.01..12.40 rows=100000 loops=1)
              Buffers: shared hit=500 read=403
Planning Time: 0.21 ms
Execution Time: 413.50 ms
```

- **ROJO** — `Seq Scan on linea_factura`: leyó los 2.000.000 de filas para quedarse con 214. `Rows Removed by Filter: 1999786`. De los 413 ms totales, 380 están en este nodo. Es el problema.
- **ROJO** — `Buffers: read=12000` en ese scan: 12.000 páginas de 8 KB (unos 94 MB) leídas desde afuera de la caché para devolver 214 filas.
- **VERDE** — `Hash Join` con `Batches: 1`: el join en sí está bien; el problema es lo que le llega.
- **VERDE** — Estimación: `rows=200` estimadas vs. 214 reales. Las estadísticas están al día. El planificador *sabía* que esto iba a ser caro; simplemente no tenía una alternativa mejor.

La solución es darle esa alternativa:

```
CREATE INDEX idx_lf_producto ON linea_factura (id_producto);
```

Plan 2 — con el índice

```
Nested Loop  (cost=0.72..1685.40 rows=200 width=16) (actual time=0.05..2.91 rows=214 loops=1)
  Buffers: shared hit=1074
  ->  Index Scan using idx_lf_producto on linea_factura l  (cost=0.43..24.93 rows=200 width=12)
                                                           (actual time=0.03..0.31 rows=214 loops=1)
        Index Cond: (id_producto = 15)
        Buffers: shared hit=217
  ->  Index Scan using factura_pkey on factura f  (cost=0.29..8.30 rows=1 width=8)
                                                  (actual time=0.01..0.01 rows=1 loops=214)
        Index Cond: (nro_factura = l.nro_factura)
        Buffers: shared hit=857
Planning Time: 0.35 ms
Execution Time: 3.02 ms
```

- **VERDE** — `Index Scan` con `Index Cond: (id_producto = 15)`: fue directo a las 214 líneas. 217 páginas leídas en lugar de 12.750.
- **VERDE** — El planificador cambió de Hash Join a `Nested Loop`: con sólo 214 filas del lado externo, conviene buscar cada factura por su clave primaria. `loops=214` y `rows=1` por vuelta: 214 filas en total,

0,01 ms cada búsqueda, unos 2 ms sumados.

- **VERDE** — `Buffers`: todo `shared hit`, nada de `read` ni `temp`.
- **Resultado:** de 413 ms a 3 ms (unas 140 veces más rápido) con exactamente las mismas 214 filas.

> **Y sin embargo…**
>
> Con el **mismo índice**, si la consulta buscara `id_producto = 1` y ese producto estuviera en el 60 % de las líneas, el planificador volvería —correctamente— al Seq Scan + Hash Join del Plan 1: 1.200.000 saltos aleatorios por índice serían mucho peores que una lectura secuencial. Mismo índice, distinto valor, distinto plan. Ese es exactamente el punto de la sección 8.

10. Checklist para leer cualquier plan

#	Paso	Qué buscar
1	**Ir al final.**	`Execution Time`: ¿es razonable para lo que la consulta devuelve? Ejecutar dos veces para descartar el efecto de la caché.
2	**Encontrar el nodo más caro.**	El que tiene mayor tiempo propio (`actual time` × `loops`, descontando sus hijos). Ahí está el problema la gran mayoría de las veces.
3	**Comparar estimado vs. real** en ese nodo.	Si `rows` difiere más de 10×: `ANALYZE tabla` y volver a probar antes de tocar otra cosa.
4	**¿Hay un Seq Scan con** `Rows Removed by Filter` **grande?**	¿Existe el índice? ¿La expresión del WHERE permite usarlo (sin funciones sobre la columna, mismos tipos, sin `LIKE '%...'`)?
5	**¿Hay algo en disco?**	`external merge Disk`, `Batches` > 1, `temp written`, `lossy`. Memoria insuficiente o demasiadas filas llegando a ese nodo.
6	**¿Hay** `loops` **enormes?**	Nested Loop o SubPlan con miles de iteraciones sobre algo que no es un Index Scan barato.
7	**Verificar la mejora.**	Aplicar *un* cambio, repetir el EXPLAIN ANALYZE, comparar `Execution Time` y las mismas filas devueltas.

11. Errores típicos al leer planes

- Leer `cost` como si fueran milisegundos. Son unidades relativas; los tiempos están en `actual time`.
- Ignorar `loops`. `rows=1 loops=100000` son 100.000 filas, y el tiempo también se multiplica.
- Concluir “Seq Scan = malo” sin mirar el tamaño de la tabla ni cuántas filas se piden.
- Probar con tablas de 10 filas y concluir que “el índice no funciona”.
- Comparar los tiempos de la primera ejecución (caché fría, mucho `read`) con los de la segunda (caché caliente).
- Correr EXPLAIN ANALYZE sobre un DELETE o UPDATE sin `BEGIN` … `ROLLBACK`.
- Crear índices para todo. Cada índice hace más lentos los INSERT, UPDATE y DELETE, y ocupa espacio. Se justifica cuando una consulta frecuente lo necesita.
- Olvidar `ANALYZE` después de una carga masiva de datos y confiar en estimaciones que ya no valen.
- Mirar sólo el nodo raíz. El tiempo total dice *cuánto* tarda; los hijos dicen *por qué*.